

Informe Técnico de Avance: Hardware, Backend y Base de Datos (Fases 0 a 4)

Proyecto: Sistema de Monitoreo Predictivo Industrial (Correa Transportadora)

Estado del Pipeline: 100% Operativo (Edge Nube API)

Fase 0: Prerrequisitos y Configuración Crítica del Entorno (WiFi)

Antes de iniciar el desarrollo en C++ para el microcontrolador, se configuró el entorno de Arduino IDE para soportar la arquitectura ESP32.

Detalle Técnico de la Librería WiFi:

Es un error común en la industria intentar descargar librerías externas llamadas “WiFi” (como WiFiNINA), lo cual genera conflictos de compilación. Para el ESP32, la librería `WiFi.h` es **nativa y viene incrustada** en el paquete base del fabricante (Espressif).

- **Paso 1:** Se agregó el repositorio oficial (https://dl.espressif.com/dl/package_esp32_index.json) en las Preferencias del IDE.
- **Paso 2:** Se descargó el Core “esp32 by Espressif Systems” desde el Gestor de Tarjetas.
- **Paso 3:** Al compilar, la directiva `#include <WiFi.h>` localiza automáticamente los archivos de red ocultos en el Core, garantizando una conexión estable sin dependencias de terceros.

Fase 1: Implementación de Hardware Real (El Edge)

Se abandonó la simulación de datos para dar paso a la adquisición física. Se realizó una auditoría de la placa **WeMos D1 R32**, descubriendo que su serigrafía utiliza la nomenclatura real de los GPIO del chip en lugar de las etiquetas genéricas de Arduino (“D1, D2...”).

Tabla de Conexiones Físicas:

Componente Físico	Función en el Monitoreo	Pin en Placa	GPIO Código
Sensor DHT22	Lectura de temperatura y humedad ambiental del motor	IO27	27
Sensor KY-002	Detección binaria de choques o vibraciones anómalas	IO25	25
Alimentación	Voltaje lógico para los sensores (No usar 5V)	3V3 / GND	N/A

Fase 2: Construcción del Backend (Node.js y MQTT)

Se desarrolló un servidor en Node.js que actúa como el “cerebro central” de recepción de datos.

- **Suscriptor MQTT:** Se creó el script `subscriber.js` que escucha activamente el tópico `planta/correa_1/telemetry` a través del broker HiveMQ.
- **Ciberseguridad Básica:** Se implementó la librería `dotenv` para ocultar las credenciales, URLs y tokens dentro de un archivo `.env`, asegurando que información sensible nunca se exponga en el código fuente.

Fase 3: Persistencia en Base de Datos (InfluxDB Cloud)

Dado el alto volumen de datos del IoT Industrial (Time-Series Data), se descartaron las bases de datos relacionales tradicionales a favor de **InfluxDB Cloud**.

- **Configuración:** Se creó el bucket `correa_transportadora` y se generó un *All Access API Token* para permitir la escritura remota.
- **Ingesta de Datos:** El archivo `subscriber.js` fue modificado para formatear la información recibida vía MQTT en “Puntos” (Points) e inyectarla a la nube.
- **Validación:** Se comprobó exitosamente la llegada de los datos reales observando la generación de gráficas en vivo mediante el *Data Explorer* nativo de InfluxDB.

Fase 4: Despliegue de la API REST Segura (Express)

Para que el futuro panel de control (Dashboard) pueda acceder a los datos sin tener las contraseñas de la base de datos, se construyó una API REST intermediaria.

- **Tecnologías:** Se utilizó el framework **Express** para el servidor web y **CORS** para permitir peticiones cruzadas de forma segura.
- **Endpoint Principal:** Se habilitó la ruta `http://localhost:3001/api/telemetry` .
- **Lógica de Consulta:** El servidor Node.js ejecuta una consulta en lenguaje **Flux**, extrayendo el historial de la última hora (`-1h`) y formateándolo en un Array JSON limpio y listo para ser consumido por el Frontend.

Resumen Arquitectónico Logrado: > Hardware Físico Microcontrolador ESP32 Wi-Fi Broker MQTT Backend Node.js Base de Datos InfluxDB API REST.

¡Disfruta tu comida! Cuando vuelvas, este documento estará aquí listo para ti, y nosotros estaremos listos para comenzar con la **Fase 5: Frontend con React**. Empezaremos a construir la parte visual e interactiva de tu planta. ¡Nos vemos a la vuelta!